Array
(C Programming Language)

Dr. Thien Huynh-The

Dept. Comp. Commun. Eng.

HCMC Technology and Education

Content

- Introduction
- Arrays
- Definition and Examples
- Passing Arrays to Functions
- Sorting Arrays
- Array Case Study
- Search Arrays
- Multiple-Subscripted Arrays

Array

- Array
  - Group of consecutive memory locations
  - Same name and type
- To refer to an element, specify
  - Array name
  - Position number
- Format:

```
arrayname[ position number ]
```

  - First element at position 0
  - `n` element array named `c`:

```
c[ 0 ], c[ 1 ]...c[ n – 1 ]
```

Array

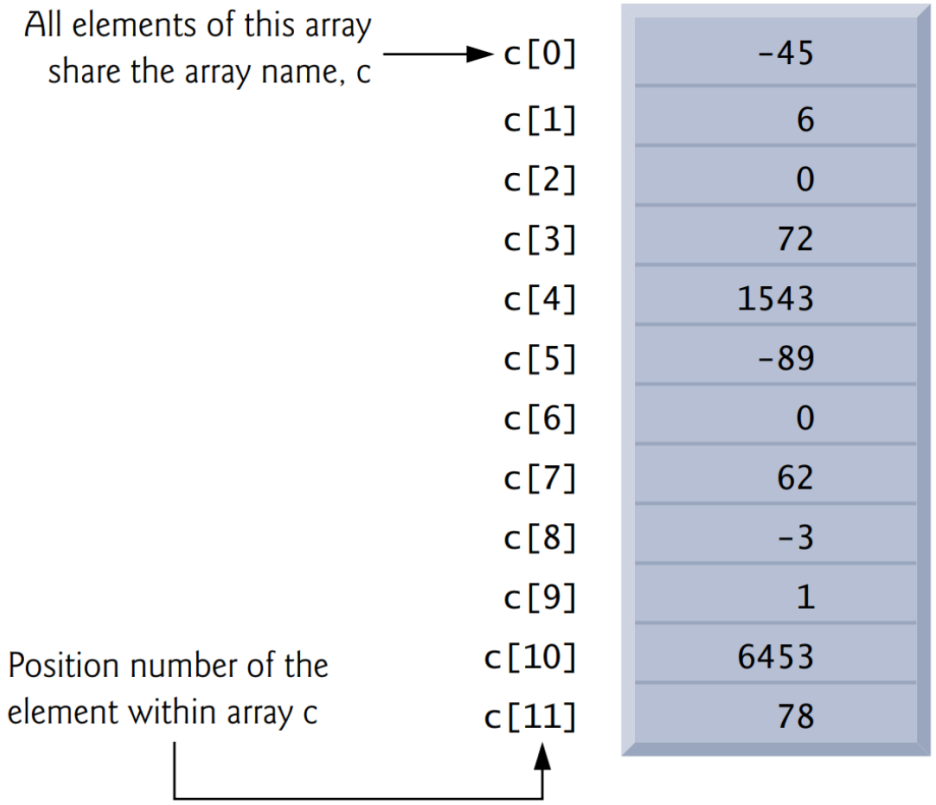

Fig. 6.1 | 12-element array.

Array

- Array elements are like normal variables

```
c[ 0 ] =  3;
printf( "%d", c[ 0 ] );
```

- Perform operations in subscript. If `x` equals 3

```
c[ 5 - 2 ] == c[ 3 ] == c[ x ]
```

Array

Operators	Associativity	Type
`[]` `()` `++` *(postfix)* `--` *(postfix)*	left to right	highest
`+` `-` `!` `++` *(prefix)* `--` *(prefix)* `(`*type*`)`	right to left	unary

Operators	Associativity	Type
`*` `/` `%`	left to right	multiplicative
`+` `-`	left to right	additive
`<` `<=` `>` `>=`	left to right	relational
`==` `!=`	left to right	equality
`&&`	left to right	logical AND
`||`	left to right	logical OR
`?:`	right to left	conditional
`=` `+=` `-=` `*=` `/=` `%=`	right to left	assignment
`,`	left to right	comma

Fig. 6.2 | Operator precedence and associativity. (Part 2 of 2.)

Array

- Defining arrays:
  - Name
  - Type of array
  - Number of elements

```
arrayType arrayName[ numberOfElements ];
```

  - Examples:

```
int c[ 10 ];
float myArray[ 3284 ];
```

- Defining multiple arrays of same type
  - Format similar to regular variables
  - Example:

```
int b[ 100 ], x[ 27 ];
```

Array

- Initializers

```
int n[ 5 ] = { 1, 2, 3, 4, 5 };
```

  - If not enough initializers, rightmost elements become 0

```
int n[ 5 ] = { 0 }
```

    - All elements 0
  - If too many initializers, a syntax error occurs
  - C arrays have no bounds checking
- If size omitted, initializers determine it

```
int n[ ] = { 1, 2, 3, 4, 5 };
```

  - 5 initializers, therefore 5 element array

Examples

- Print element values in an arrays

```
#include <stdio.h>
int main(){
    int arrayA[5]={1,2,3,4,5};
    for (int i=0; i<5; ++i)
            printf("arrayA[%d]=%d\n", i, arrayA[i]);
    return 0;
}
```

Examples

- Read element values in an arrays

```
#include <stdio.h>
int main(){
    int arrayA[5];
    for (int i=0; i<5; ++i){
            printf("\n arrayA[%d]= ",i);
            scanf("%d", &arrayA[i]);
    }
    return 0;
}
```

Exercises

- Viết chtrình theo yêu cầu sau:
  - Nhập vào giá trị cho n phần tử mảng arrayA
  - Tính tổng các giá trị trong mảng arrayA

Examples

- Write a program to calculate the sum of all elements in an array

```
#include <stdio.h>
int main(){
    int arrayA[5], sum=0;
    for (int i=0; i<5; ++i){
            printf("\n arrayA[%d]= ",i);
            scanf("%d", &arrayA[i]);
    }
    for (int i=0; i<5; ++i)
            sum += arrayA[i];
    printf("Sum of all elements in an array: %d", sum);
}
```

Exercises

- Write a program to calculate the power of 2 raised to elements in an array arrayA[5]={1,2,3,4,5}

- Write a program to calculate the sum of each element of arrayA[5]={1,3,5,7,9} and arrayB[5]={2,4,6,8,10}

- Write a program to input 20 elements for an arrayA by a user
  - Find and print all prime numbers in the arrayA
  - Find and print all numbers dividiable by 3 in the arrayA
  - Find and print all perfect square numbers in the arrayA
  - Order all elements in the array A from smallest to largest

Exercises

- Write a program with the following requirements:
  - Create two arrays, denoted arrayA and arrayB, that have 10 random numbers
  - Calculate the sum of each element of arrayA and arrayB, and store in a new array C
  - Calculate the multiplication of each element of arrayA and arrayB, and store in a new arrayD

- Write a program with the following requirements:
  - Create an array with 100 random numbers
  - Find and print all prime numbers in the array
  - Find and print all perfect square numbers in the array
  - Find and print all number dividible by 3 and 5 simultaneously in the array

Multi-dimensional Array

- How to declare?

```
int arrayA[3][4]
float arrayB[2][4][3]
```

- Initializing a multi-dimensional arrays

```
int arrayA[2][3]={{1, 3, 0},{-1, 5, -9}};
int arrayA[][3]={{1, 3, 0},{-1, 5, -9}};
int arrayA[2][3]={1, 3, 0, -1, 5, -9};
```

- Initializing a 3D array

```
int arrayA[2][3][4]={{{3, 4, 2, 3}, {0, -3, 9, 11}, {23, 12, 21, -4}},
{{13, 4, 56,3},{5, 9, 3, 5} ,{3, 1, 4, 9}}};
```

Multi-dimensional Arrays

- Write a program to calculate the sum of two matrices

```
#include <stdio.h>
int main(){
    float a[2][2], b[2][2], sum[2][2];
    printf("Enter elements of 1st matrix \n");
    for (int i=0; i<2; ++i)
            for (int j=0; j<2; ++j){
                    printf("Enter a[%d][%d]: ", i+1, j+1);
                    scanf("%f", &a[i][j]);}
    for (int i=0; i<2; ++i)
            for (int j=0; j<2; ++j){
                    printf("Enter b[%d][%d]: ", i+1, j+1);
                    scanf("%f", &b[i][j]);}
```

Multi-dimensional Arrays

```
        // adding corresponding elements of two arrays
        for (int i=0; i<2; ++i)
                for (int j=0; j<2; ++j)
                        sum[i][j]=a[i][j]+b[i][j];
        printf("\nSum of matrices: \n");
        for (int i=0; i<2; ++i)
                for (int j=0; j<2; ++j){
                        printf("%0.1f\t", sum[i][j]);
                        if (j==1)
                                puts("");
                        }
        return 0;
}
```

Character Arrays

- Character arrays
  - String “`first`” is really a static array of characters
  - Character arrays can be initialized using string literals

```
char stringA[] = "first";
```

    - Null character '`\0`' terminates strings
    - `stringA` actually has 6 elements
    - It is equivalent to

```
char stringA[] = { 'f', 'i', 'r', 's', 't', '\0' };
```

  - Can access individual characters

`stringA[ 3 ]` is character ‘`s`’

  - Array name is address of array, so & not needed for scanf

```
scanf( "%s", stringB );
```

  - Reads characters until whitespace encountered

Character Arrays

- Declare a string

```
char stringA[5];
```

- Initialize strings

```
char stringA[]="abcd";
char stringA[50]="abcd";
char stringA[]={'a','b','c','d','\0'};
char stringA[5]={'a','b','c','d','\0'};
```

- Notes

```
char c[5]="abcde";
```

- We are trying to assign 6 characters (the last character is) to a char array having 5 characters. This is bad and you should never do this

Character Arrays

- Arrays and string are second-class citizens in C
- They do not support the assignment operator once it is declared

```
char c[100];
c = "C programming"; //Error! array type is not assignable
```

Character Arrays

- Read string from the user

```
#include <stdio.h>
int main(){
        char city[20];
        printf("Enter your city: ");
        scanf("%s", city);
        printf("Your city is %s.", city);
        return 0;
}
```

- Even though Ho Chi Minh was entered in the above program, only Ho was stored in the name string because there was a space after Ho.

Character arrays

- How to read a line of text
- Use the `fgets()` function to read a line of string.
- You can use `puts()` to display the string

```
#include <stdio.h>
int main(){
    char name[30];
    printf("Enter name: ");
    fgets(name, sizeof(name), stdin); //read string
    printf("Name: ");
    puts(name); //display string
    return 0;}
```

Examples

```
#include <stdio.h>
int main (){
    char stringA[20];
    char stringB[] = "C programming language";
    int i;
    // read string from user into array stringA
    printf("Enter a string: ");
    scanf("%s", stringA);
    //output strings
    printf("stringA is: %s\nstringB is: %s\n"
    "stringA with spaces between characters is: \n", stringA,  stringB);
    // output characters until nill character is reached
    for (i=0; stringA[i]!='\0';i++){
            printf("%c ", stringA[i]);}
    printf("\n");
    return 0;}
```

Examples

Static array example

```
#include <stdio.h>
void staticArrayInit(void);
void automaticArrayInit(void);
int main(void){
    printf("First call to each function: \n");
    staticArrayInit();
    automaticArrayInit();

    printf("\n\nSecond call to each function:\n");
    staticArrayInit();
    automaticArrayInit();
    return 0;}
```

Examples

```
void staticArrayInit(void){
    static int arrayA[3];
    int i;
    printf("\nValues on entering staticArrayInit:\n");
    //output contents of arrayA
    for (i=0; i<=2; ++i)
           printf("arrayA[%d] = %d ", i, arrayA[i]);
    printf("\nValues on existing staticArrayInit:\n");
    //modify and ouput contents of arrayA
    for (i = 0; i<=2; ++i)
           printf("arrayA[%d] = %d ", i, arrayA[i]+=5);}
```

Examples

```
void automaticArrayInit(void){
    int arrayB[3]={1,2,3};
    int i;
    printf("\nValues on entering automaticArrayInit:\n");
    //output contents of arrayA
    for (i=0; i<=2; ++i)
           printf("arrayB[%d] = %d ", i, arrayB[i]);
    printf("\nValues on existing staticArrayInit:\n");
    //modify and ouput contents of arrayA
    for (i = 0; i<=2; ++i)
           printf("arrayB[%d] = %d ", i, arrayB[i]+=5);}
```

Character Arrays

- Count a letter in a string

```
#include <stdio.h>
int main(){
    char s[10]="Hello";
    int dem=0;
    for (int i=0; i<10; ++i)
            if (s[i]=='l')
                    dem++;
    printf("So ky tu l trong chuoi: %d", dem);
    return 0;
}
```

Process Strings

- Function atof – convert a string from char to double

```
double atof (const char *s)
```

- Examples

```
#include <stdio.h>
#include <stdlib.h>
int main(){
    float f;
    char stringA[]="1234.56";
    f=atof(stringA); //
    printf("%s \n", stringA);
    printf("%0.2f", f+10);}
```

Process Strings

- Function atoi – convert a string from char to int

```
int atoi (const char *s)
```

- Examples

```
#include <stdio.h>
#include <stdlib.h>
int main(){
    int i;
    char stringA[]="1234";
    i = atoi(stringA);
    printf("%s \n", stringA);
    printf("%d", i+10);}
```

Process Strings

- Function strlen – return the length of a string

```
int strlen (const char *s)
```

- Examples

```
#include <stdio.h>
#include <string.h>
int main(){
    int stringLen;
    char stringA[]="Hello world!";
    stringLen = strlen(stringA);
    printf("%d", stringLen);}
```

Process Strings

- Function strcpy – copy a string to another variable

```
int strcpy (char *string, stringA)
```

- Examples

```
#include <stdio.h>
#include <string.h>
int main(){
    char stringA[]="Hello world!";
    char stringB[30];
    strcpy(stringB, stringA);
    printf("%s", stringB);}
```

Process Strings

- Function tolower – convert a string from uppercase to lowercase

```
int tolower (int char *s)
```

- Examples

```
#include <stdio.h>
#include <string.h>
int main(){
    char stringA[]="HELLO WORLD";
    int len = strlen(stringA);
    for (int i=0; i<len; ++i)
            stringA[i]=tolower(stringA[i]);
    printf("%s", stringA);}
```

Process Strings

- Function tolower – convert a string from lowercase to uppercase

```
int toupper (int char *s)
```

- Examples

```
#include <stdio.h>
#include <string.h>
int main(){
    char stringA[]="hello world";
    int len = strlen(stringA);
    for (int i=0; i<len; ++i)
            stringA[i]=toupper(stringA[i]);
    printf("%s", stringA);}
```

Process Strings

- Function tolower – combine 2 or more strings into a string

```
char strcat(char *string, const char *stringA)
```

- Examples

```
#include <stdio.h>
#include <string.h>
int main(){
    char s[30]="";
    char s1[2]=" ", s2[6]="hello", s3[7]="world!";
    strcat(s,s2);
    strcat(s,s1);
    strcat(s,s3);
    printf("%s", s);
    return 0;}
```

Exercises

- Write a program to create a 3D array, denoted arrayA, with random number in [1,100]
  - Print the 3D arrayA
  - Find and print all prime numbers in arrayA
  - Find and print all perfect square numbers in arrayA
  - Calculate the results when passing the array to a function

$$f(x) = \begin{cases} 0.5x & ; \text{if } x < 30 \\ x^2 - 5x & ; \text{if } 30 \leq x < 65 \\ \frac{(x^2-2)}{x+4} & ; \text{if } x \geq 65 \end{cases}$$

Passing Array to Function

- Function prototype

```
void modifyArray(int b[]), int arraySize);
```

- Parameter names optional in prototype (not recommend)

  `int b[]` could be written `int []`

  `int arraySize` could be simply `int`

Passing Array to Function

- Passing individual array elements

```
#include <stdio.h>
void display (int number1, int number2){
    printf("%d\n", number1);
    printf("%d\n", number2);}
int main(){
    int arrayA[]={2,8,4,12};
    // pass second and third elements to display()
    display(arrayA[1], arrayA[2]);
    return 0;}
```

- Write a short program to check whether an element in an array is the prime number or not

Passing Array to Function

- Passing an array with all elements
- Find and print the minimum value in an array

```
#include <stdio.h>
int minarray(int array[], int size);
int main(){
    int i=0, min=0;
    int numbers[]={4,5,6,2,1,6,10};
    min=minarray(numbers, 7);
    printf("minimum number is %d\n", min);
}
int minarray(int array[], int size){
    int min=array[0];
    for (int i=0; i<size; ++i)
            if (min>array[i])
                    min=array[i];
    return min;}
```

Passing Array to Function

- Passing an array with all elements
- Calculate the sum of all values in an array

```
#include <stdio.h>
int minarray(int array[], int size);
int main(){
    int sum;
    int arrayA[]={4,5,6,2,1,6,10};
    sum=sumArray(arrayA, 7);
    printf("Sum of all elements is %d\n", sum);
}
int sumArray(int array[], int size){
    int sum=0;
    for (int i=0; i<size; ++i)
            sum +=array[i];
    return sum;}
```

Sorting Array

- Sorting data
  - Important computing application
  - Virtually every organization must sort some data
- Bubble sort
  - Several passes through the array
  - Successive pairs of elements are compared
    - If increasing order (or identical) -> no change
    - If decreasing order -> elements exchanged
  - Repeat
- Example
  - Original: 3 4 2 6 7
  - Pass 1: 3 2 4 6 7
  - Pass 2: 2 3 4 6 7
  - Small elements “bubble” to the top

Bubble Sort

```
#include <stdio.h>
#define SIZE 10
int main(){
        int a[SIZE]={2,6,4,8,5,7,89,23,54,100};
        int pass, hold;
        printf("Data items in original order \n");
        for (int i=0; i<SIZE; ++i)
                printf("%4d", a[i]);
        for (pass=1; pass<SIZE; ++pass){
                for (int i=0; i<SIZE-1; ++i)
                        if(a[i]>a[i+1]){
                                hold=a[i];
                                a[i]=a[i+1];
                                a[i+1]=hold;}
        }
        printf("\nData items in ascending order\n");
        for (int i=0; i<SIZE; ++i)
                printf("%4d", a[i]);
        return 0;}
```

Case Study

- Computing mean, median, and mode using arrays
  - Mean – average
  - Median – number in middle of sorted list

    1, 2, 3, 4, 5

    3 is the median value
  - Mode – number that occurs most often

    1, 1, 1, 2, 3, 3, 4, 5

Mean of An Array

```
#include <stdio.h>
#define SIZE 30
void mean(const int answer[]);
void median(int answer[]);
void mode(int freq[], const int answer[]);
void bubbleSort(int array[]);
void printArray(const int array[]);
int main(){
        int frequency[10]={0};
        int
response[SIZE]={3,4,5,6,3,4,5,8,2,4,4,7,4,1,2,9,7,4,2,1,1,1,2,3,4,9,2,3,4,8};
        mean(response);}
void mean(const int answer[]){
        int total=0;
        for (int j=0; j<SIZE; ++j)
                total +=answer[j];
        printf("the mean value of array: %f", (float) total/SIZE);}
```

Median of An Array

```
#include <stdio.h>
#define SIZE 30
void median(int answer[]);
void bubbleSort(int array[]);
void printArray(const int array[]);
int main (){
            int frequency[10]={0};
            int response[SIZE]={3,4,5,6,3,4,5,8,2,4,4,7,4,1,2,9,7,4,2,1,1,1,2,3,4,9,2,3,4,8};
            median(response);}
void median(int answer[]){
            printArray(answer);
            bubbleSort(answer);
            printf("\n\nThe sorted array is ");
            printArray(answer);
            printf("\nThe median element is %d", answer[SIZE/2]);}
void bubbleSort(int array[]){
            int pass, i, hold;
            for (pass=1; pass<SIZE; ++pass)
                        for (i=0; i<SIZE-1; ++i)
                                    if (array[i]>array[i+1]){
                                                hold=array[i];
                                                array[i]=array[i+1];
                                                array[i+1]=hold;}}
void printArray(const int array[]){
            for (int i=0; i<SIZE; ++i)
                        printf("%d ", array[i]);}
```

Mode of An Array

```
#include <stdio.h>
#define SIZE 30
void mode(int freq[], const int answer[]);
int main(){
            int frequency[10]={0};
            int response[SIZE]={3,4,5,6,3,4,5,8,2,4,4,7,4,1,2,9,7,4,2,1,1,1,2,3,4,9,2,3,4,8};
            mode(frequency, response);
            return 0;}
void mode(int freq[], const int answer[]){
            int rating, i, h, largest=0, modeValue=0;
            for (rating=0; rating<=9; ++rating)
                        freq[rating]=0;
            for (i=0; i<SIZE; ++i)
                        ++freq[answer[i]];
            for (rating=1; rating<=9; ++rating){
                        printf("%8d%11d                    ", rating, freq[rating]);
                        if (freq[rating]>largest){
                                    largest = freq[rating];
                                    modeValue = rating;}
                        for (h=1; h<=freq[rating]; ++h)
                                    printf("*");
                        printf("\n");           }
            printf("The mode of the most frequency value. \nFor this run the mode is %d which occured %d time. \n", modeValue,
largest);}
```

Exercises

- Write a complete program with the following requirements:
  - Enter one integer array with n elements (n is inputted by a user)
  - Enter one integer X (X is inputted by a user). Count the number of elements of the array in which values of these elements are less than X
  - Sort and export arrays in descending order
  - Export the even elements in the array in descending order
  - Export the odd elements in the array in ascending order

Searching Array

- Binary search
  - For sorted arrays only
  - Compares middle element with key
    - If equal, match found
    - If key < middle, looks in first half of array
    - If key > middle, looks in last half
    - Repeat
  - Very fast: at most n steps, where 2n > number of elements
    - 30 elements array takes at most 5 steps
    - $2^5 > 30$ so at most 5 steps

Examples

- Read the code example 6.19 in pages 275-278 and understand it `<binary search of a sorted array>`
- Read the code example 6.22 in pages 282-284 and understand it `<two-dimensional array multiplications>`

Exercises

- Write a program to delete all duplicate numbers in an array
- Write a program to merge two sorted array
- Write a program to put even and odd numbers of an array into two separate arrays
- Write a program to find and print the reverse of an array
- Write a program to left and right rotate an array
- Write a program to calculate the sum of diagonal elements of an 2D array (or a matrix)

Exercises

- Write a program to find the transpose of a given 2D array (entered by user)
- Write a program to calculate the sum of each row and column of a given 2D array (entered by user)
- Write a program with the functions to find the frequency of odd and even number in a given 2D array (entered by user)
- Write a program to interchange diagonals of a given 2D array (entered by user)